

C++ STYLE GUIDE

Programming Guidelines

Functions

- Care should be taken to try to keep functions to less than the height of a single screen (usually 30–35 lines). If a function is more than 35 lines, you should consider trying to break the function into smaller functions.

Note: this is not a strict rule. Sometimes a function will have to be more than 35 lines. However, you should try to keep functions as short and concise as possible.

- If the exact same lines of code are repeated twice or more in a program, that code should be moved into a function.

Note: single line statements such as returns or variable assignments are excluded from this rule.

Variable declarations

- Instance variables should be declared at the top of a class/struct.
- A variable should always be given an initial value unless you have a very good reason not to.
- If a number, word, or piece of data is declared in more than two places in a function or group of functions then it should be made into a variable.

Note: if this piece of data will never change then the variable should be made constant. If a constant is used in more than one function, it should be declared as a global constant (after preprocessor includes, but before all functions).

White space (indentation, braces, and new lines)

- White space must be used to make code more readable.
- (not a strict rule) Declarations of local variables should be separated from the body of a function by a new line when possible.
- NEVER use non-constant global variables.
- NEVER use a non-constant variable to give the size of an array.

Indentation

- Every block gets one additional indent level relative to the line that opened it. This applies to all blocks, including single-line scopes (see the scope diagram at the end of this guide).
- This guide does not mandate spaces vs. tabs, since everyone may be using a different editor. Whichever you use, be consistent within a file, and do not mix tabs and spaces in the same file — mixing the two is what causes code to look correctly indented in one editor and broken in another.

Continuation lines

A continuation line is a line that continues a single statement onto the next line, rather than starting a new block — for example, a function call or definition with too many arguments to fit on one line, or a long condition. Indent continuation lines one extra level beyond the current block, so they're visually distinguishable from the start of a new nested block:

```
int total = calculateOrderTotal(unitPrice, quantity,
                                discountRate, taxRate);

if (accountIsActive && balance > minimumBalance &&
    !accountIsFlagged) {
```

```
    processTransaction();  
}
```

Brace placement

Braces are required around every block, with no exceptions — including single-statement if/else/for/while bodies. Never omit the braces just because a block is one line long.

Pick one of the two styles below for a given project and use it consistently throughout that project. Do not mix styles within the same codebase.

Option 1: K&R style

The opening brace stays on the same line as the statement that starts the block. The closing brace gets its own line.

```
if (myBool) {  
    doSomething();  
} else {  
    doSomethingElse();  
}  
  
while (myBool) {  
    doSomething();  
}
```

Option 2: Allman style

The opening brace goes on its own line, aligned with the line that starts the block. The closing brace also gets its own line, aligned with the opening brace.

```
if (myBool)  
{  
    doSomething();  
}  
else  
{  
    doSomethingElse();  
}  
  
while (myBool)  
{  
    doSomething();  
}
```

Comments

- Comments are used in order to inform programmers what the code is doing.
- Comments that span more than three lines should use the multi-line comment `/* */` rather than the single-line `//` comment.
- All functions other than `main()` must have a comment above them explaining what the function does and its role in the program.

Note: You do not have to include a comment above functions within a class so long as the name of the function clearly describes what it does.

- There must be a multi-line comment at the top of the file containing `main()` consisting of your name, the question the code responds to, and a description of what the program does.
- For files declaring classes (.h files) include a multi-line comment at the top of the file containing your name, the question the code responds to, and a description of the role of the class in your program.

- You should do your best to convey what a piece of code is doing by using good variable and function names, but additional comments should be added when a piece of code's function is ambiguous or when a piece of code must perform a complex task.

Custom data type declarations (classes, structs, typedefs, etc.)

- Generally classes, structs, and objects that have instance variables should be declared in their own file, unless they are utilized by only a single function or small group of functions in the same file.
- Generally enums and typedefs that are used for a single function or group of functions can be declared alongside said functions if they are all in the same file. Declare typedefs and enums above all functions, but below preprocessor include directives.

If and Switch statements

- Care should be taken to avoid nesting if or switch statements greater than two levels down.
Note: sometimes if/switch statements will have to be nested inside each other. Nevertheless, it should be avoided if possible.

Loops

- Care should be taken to avoid nested loops greater than two levels down.
Note: sometimes loops will have to be nested inside each other, but it should be avoided if possible.

User input

- (Later in the semester) input should be screened for simple type mismatches (i.e. letters inside of numbers).
- User input should be screened for out-of-bounds inputs.
Note: input should be screened for simple wrong inputs, but don't become obsessed with trying to catch every single wrong input.

Naming Conventions

General naming convention

- In general, names (function names, variable names) should convey the use of a piece of code. If you can't figure out any sensible name whatsoever for a variable or function, this indicates deep problems in your program design — figure out how to fix these flaws on paper before writing more code.
- Do not use single-character variable or function names.

Function and variable names

NOTE: these guidelines give you an option of using camelCase or snake_case. Whichever you choose, PLEASE USE IT CONSISTENTLY. It will be considered terrible style if you mix camelCase and snake_case in the same program.

- Function and variable names should be in camelCase or snake_case.
 - camelCase: first word starts with lowercase and following words start with uppercase. Example: thisIsSomeCamelCase
 - snake_case: all lowercase, underscore between words.
- Function names should give a good indication of a function's usage.
- Constants should be in ALL CAPITALS and use snake_case.

- Non-constant global variables should not be used. However, if you run into a case where one is ABSOLUTELY necessary they should start with `g_` and then be camelCase or snake_case.
- It's worth repeating: Non-constant global variables should not be used. If it is absolutely necessary to use a global variable, please include a comment above the variable explaining why it is necessary. "I'm using the variable in more than one function" is not a good explanation — in this case, define the functions to take the variable as a parameter. The use of global variables makes it much, much more difficult for you to reuse functions in multiple programs, makes it unclear which functions act on what variables, and is considered terrible programming practice. (In software development jargon, globals reduce the cohesion of your program.)
- Local variables should use camelCase or snake_case.
- Variable names should give a good indication of what the variable is used for and what type of data it stores.
- While designing classes, take care not to use unnecessary instance variables. If it is possible to move an instance variable into one of the functions defined within the class, please do so.

Custom data types

- Data type names (classes, structs, enums, typedefs) should use PascalCase.
 - PascalCase: the first letter of each word is capitalized. Example: ThisIsSomePascalCase

Scope diagram

The diagram illustrates indentation and scope nesting — variable scope, not brace placement. It happens to use Allman-style braces; the same nesting/indentation logic applies whichever brace style your project has chosen.

```
const int myConstant = 10;    // global scope

returnType myFunction()
{    // function scope
    bool myBool = true;
    char myChar = 'A';

    // ^^ separate declarations from body ^^
    while (myBool)    // comment on what this does
    {    // loop scope 1
        for (int counter = 0; counter < myConstant; counter++)
        {    // loop scope 2
            if (myChar == 'B')
            {    // if scope
                myBool = false;
                if (myConstant == 4)
                {
                    myBool = true;    // single line scope
                }
            }
            else if (myChar == 'A')
            {    // another if scope
                myChar = 'B';
            }
        }
    }
}
```